

**King Fahd University of Petroleum & Minerals**  
**Information & Computer Science Department**  
**ICS-202: Data Structures and Algorithms**

---

## **PROJECT GUIDELINES**

This document provides a guideline for ICS-202 Lab project. The students in the group are required to arrange a team meeting by themselves and distribute the tasks evenly.

**TOTAL MARKS:**

30 scaled down to 5

**DEADLINE TO SUBMIT THE TEAM:**

October 23, 2025

**PROJECT SUBMISSION:**

Students are required to submit the project's product one day before the demo in Week 15.

**PROJECT DEMO (10-15 MINS):**

- Project demo will be held in Week 15 during the regular class time.
- All the members of the group are required to give a live demonstration of the project code. (Divide the material equally).
- Each member of the group will run and explain a specific section of the code.

## Lab Project Tasks

# TermFS - A Terminal-Based File System Simulator

### 1. Project Objectives

This project aims to solidify your understanding of fundamental and advanced data structures by designing and implementing a simulated in-memory file system. You will build a command-line interface that mimics common UNIX shell commands. This project will require you to:

- Design and implement a hierarchical class structure using inheritance and polymorphism.
- Represent the file system hierarchy (directories and files).
- Implement efficient file and directory lookups within a directory.
- Manage path navigation.
- Implement file content searching.
- Write functions for operations that traverse the file system tree.

### 2. Problem Statement

You are to create a program that simulates a simple file system that exists only in memory. The program will start with a single root directory, `/`. Users will interact with this file system by typing commands into a prompt that shows the current working directory (e.g., `/home/user$`). Your simulator must parse these commands and modify the state of the file system accordingly.

The core of your project will be the data structures you choose to represent the file system entities. A directory can contain multiple files and other directories, forming a tree-like structure. The efficiency of your operations, such as looking up a file in a large directory, will depend directly on your choice and implementation of the underlying data structures.

### 3. Functional Requirements

- Your file system must be represented by a hierarchical tree structure.
- An abstract class or interface **Node** should define common properties for files and directories (e.g., name, parent directory).
- Concrete classes **File** and **Directory** should inherit from **Node**. This will allow you to use polymorphism for operations like calculating size.

- Each **Directory** object must use a Hash Table to store its children. The key should be the child's name (String), and the value should be the **Node** object (the file or subdirectory).
- The simulator must support the following commands:

| Command                                                 | Description                                                                                                                                                                                                                                                         | Data Structures/Algorithms Involved |
|---------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------|
| <code>mkdir &lt;dir_name&gt;</code>                     | Creates a new directory in the current directory. It can be followed by <code>-p</code><br><code>-p</code> : This is an option (a flag) that stands for "parents" or "path". It tells <code>mkdir</code> to create any parent directories that don't already exist. | Tree Manipulation, Hashing          |
| <code>touch &lt;file_name&gt; &lt;size&gt;</code>       | Creates a new empty file with a given size.                                                                                                                                                                                                                         | Tree Manipulation, Hashing          |
| <code>echo "&lt;text&gt;" &gt; &lt;file_name&gt;</code> | Writes <code>&lt;text&gt;</code> to the specified file, creating it if it doesn't exist or overwriting it if it does.                                                                                                                                               | Tree Manipulation, Hashing          |
| <code>ls</code>                                         | Lists the contents of the current directory.                                                                                                                                                                                                                        | Hash Table Iterator                 |
| <code>cd &lt;path&gt;</code>                            | Changes the current directory. Supports <code>..</code> (parent), absolute paths (e.g., <code>/home/user</code> ), and relative paths (e.g., <code>mydir</code> ).                                                                                                  | Stack, Tree Traversal               |
| <code>pwd</code>                                        | Prints the full path of the current working directory.                                                                                                                                                                                                              | Stack                               |
| <code>rm &lt;name&gt;</code>                            | Removes a file or an empty directory.                                                                                                                                                                                                                               | Hashing, Tree Manipulation          |
| <code>rm -r &lt;dir_name&gt;</code>                     | Recursively removes a directory and all its contents.                                                                                                                                                                                                               | Recursion, Tree Traversal           |
| <code>tree</code>                                       | Displays the entire file system structure starting from the current directory.                                                                                                                                                                                      | Recursion / Queue                   |
| <code>grep "&lt;pattern&gt;" &lt;file_name&gt;</code>   | Searches for the <code>&lt;pattern&gt;</code> inside the specified file using the KMP algorithm.                                                                                                                                                                    | String Matching                     |
| <code>du</code>                                         | Displays the total size of the current directory and all its contents (disk usage).                                                                                                                                                                                 | Recursion, Tree Traversal           |

### Notes:

1. You should draw text-based trees as shown in sample output when the command 'tree' is given (using the lines shown below, copy them if needed for coding)

- For a non-last item: `|—`
- For the last item: `|_`
- For a vertical line: `|`

2. In-Memory vs. Real File System: What you'll be building is a pure simulation—a self-contained virtual environment. In-Memory Implementation means that your entire file system, including all its directories, files, and their contents, will exist exclusively in the computer's RAM (Random Access Memory) while your program is running. You will use data structures like classes, objects, trees, and hash tables to represent it. Your program will not read from or write to your actual PC's file system (like your C: drive or home directory). Commands like `mkdir` in your simulator will create a `Directory` object in memory, not a real folder on your hard drive.
3. Initial State: At the very beginning, when the program first launches, the file system will be almost empty. The only thing that will exist is the root directory, represented as `/`. This is the top-level node of your tree structure, from which all other directories and files will be created using commands like `mkdir` and `touch`.
4. File Size Simulation: The creation of a file with a specific size is also a simulation. When you execute a command like `touch report.txt 512`, your program will not allocate 512 bytes of real disk space. Instead, it will likely do the following: Create an instance of your `File` class. Set its `name` property to "report.txt". Set its `size` property (an integer variable inside the class) to `512`. When another command like `du` or `ls` needs to know the file's size, it will simply read this integer variable from the File object. Similarly, for the `echo` command, the "size" would just be the length of the string content stored in a variable within the `File` object.
5. Use comments in your code as much as you can.

## 4. Sample Input and Output

Your program will read commands and produce output to the console. The prompt should display the current path followed by a '\$'.

### *Sample 1: Basic File and Directory Creation*

#### **Input:**

```
mkdir home
ls
cd home
mkdir user
touch profile 15
ls
cd ..
ls
```

#### **Output:**

```
/$ mkdir home
/$ ls
home/
/$ cd home
/home$ mkdir user
/home$ touch profile 15
/home$ ls
user/  profile (15B)
/home$ cd ..
/$ ls
home/
```

### *Sample 2: Path Navigation and pwd*

#### **Input:**

```
mkdir -p a/b/c
cd a/b/c
pwd
cd ..
```

```
pwd
cd /
pwd
```

**Output:**

```
/$ mkdir -p a/b/c
/$ cd a/b/c
/a/b/c$ pwd
/a/b/c
/a/b/c$ cd ..
/a/b$ pwd
/a/b
/a/b$ cd /
/$ pwd
/
```

**Sample 3: tree and du commands**

**Input:**

```
mkdir docs
cd docs
mkdir reports
touch final_report.txt 1024
echo "first draft" > draft.txt
cd reports
touch Q1.txt 512
cd /
tree
du
cd docs
du
```

**Output:**

```
/$ mkdir docs
/$ cd docs
/docs$ mkdir reports
/docs$ touch final_report.txt 1024
/docs$ echo "first draft" > draft.txt
/docs$ cd reports
/docs/reports$ touch Q1.txt 512
/docs/reports$ cd /
/$ tree
.
└─ docs/
```

```
├─ reports/
│   └─ Q1.txt (512B)
├─ final_report.txt (1024B)
└─ draft.txt (11B)
/$ du
Total size: 1547B
/$ cd docs
/docs$ du
Total size: 1547B
```

#### ***Sample 4: grep (KMP) Successful Match***

##### **Input:**

```
echo "The quick brown fox jumps over the lazy dog" > sentence.txt
grep "brown fox" sentence.txt
grep "lazy" sentence.txt
```

##### **Output:**

```
/$ echo "The quick brown fox jumps over the lazy dog" > sentence.txt
/$ grep "brown fox" sentence.txt
Pattern "brown fox" found in sentence.txt.
/$ grep "lazy" sentence.txt
Pattern "lazy" found in sentence.txt.
```

#### ***Sample 5: grep No Match***

##### **Input:**

```
echo "The quick brown fox jumps over the lazy dog" > sentence.txt
grep "red fox" sentence.txt
```

##### **Output:**

```
/$ echo "The quick brown fox jumps over the lazy dog" > sentence.txt
/$ grep "red fox" sentence.txt
Pattern "red fox" not found in sentence.txt.
```

### ***Sample 6: rm and rm -r***

#### **Input:**

```
mkdir temp
cd temp
touch file1 10
mkdir sub
cd sub
touch file2 20
cd /
rm temp
rm -r temp
ls
```

#### **Output:**

```
/$ mkdir temp
/$ cd temp
/temp$ touch file1 10
/temp$ mkdir sub
/temp$ cd sub
/sub$ touch file2 20
/sub$ cd /
/$ rm temp
Error: Cannot remove directory 'temp'. It is not empty.
/$ rm -r temp
/$ ls
(empty output)
```

### ***Sample 7: Edge Cases and Error Handling***

#### **Input:**

```
touch myfile 1
cd myfile
mkdir existing
mkdir existing
rm non_existent_file
cd /no/such/path
```

#### **Output:**



```
/$ touch myfile 1
/$ cd myfile
Error: 'myfile' is not a directory.
/$ mkdir existing
/$ mkdir existing
Error: 'existing' already exists.
/$ rm non_existent_file
Error: 'non_existent_file' not found.
/$ cd /no/such/path
Error: Path '/no/such/path' not found.
```

## Sample 8: Large Complex Structure

### Input:

```
mkdir bin etc home var
cd home
mkdir alice
mkdir bob
cd alice
mkdir Desktop Documents Downloads
echo "project plan details" > Documents/plan.doc
touch Downloads/archive.zip 50000
cd /etc
touch config.conf 2048
mkdir network
echo "eth0: active" > network/interfaces
cd /var
mkdir log
touch log/system.log 3000
cd /
tree
```

### Output:

```
/$ mkdir bin etc home var
/$ cd home
/home$ mkdir alice
/home$ mkdir bob
/home$ cd alice
/home/alice$ mkdir Desktop Documents Downloads
/home/alice$ echo "project plan details" > Documents/plan.doc
/home/alice$ touch Downloads/archive.zip 50000
/home/alice$ cd /etc
/etc$ touch config.conf 2048
/etc$ mkdir network
/etc$ echo "eth0: active" > network/interfaces
/etc$ cd /var
```

```

/var$ mkdir log
/var$ touch log/system.log 3000
/var$ cd /
/$ tree
.
├── bin/
├── etc/
│   ├── config.conf (2048B)
│   └── network/
│       └── interfaces (12B)
├── home/
│   ├── alice/
│   │   ├── Desktop/
│   │   ├── Documents/
│   │   │   └── plan.doc (20B)
│   │   └── Downloads/
│   │       └── archive.zip (50000B)
│   └── bob/
└── var/
    └── log/
        └── system.log (3000B)

```

### *Sample 9: Complex du and rm -r Scenario*

#### **Input:**

```

(Commands from Sample 8 are assumed to have been run)
du
cd home
du
cd alice
rm -r Downloads
du
cd /
du

```

#### **Output:**

```

...
/$ du

```

```
Total size: 55080B
/$ cd home
/home$ du
Total size: 50020B
/home$ cd alice
/home/alice$ rm -r Downloads
/home/alice$ du
Total size: 20B
/home/alice$ cd /
/$ du
Total size: 5060B
```

#### *Sample 10: Complex grep with Overlapping Patterns*

##### **Input:**

```
echo "ababababca" > test_kmp.txt
grep "abababca" test_kmp.txt
grep "ababca" test_kmp.txt
grep "ac" test_kmp.txt
```

##### **Output:**

```
/$ echo "ababababca" > test_kmp.txt
/$ grep "abababca" test_kmp.txt
Pattern "abababca" found in test_kmp.txt.
/$ grep "ababca" test_kmp.txt
Pattern "ababca" found in test_kmp.txt.
/$ grep "ac" test_kmp.txt
Pattern "ac" not found in test_kmp.txt.
```

**Submission Details:**

- Submit all Java files.
- Submit a “readme.txt” file which should contain instructions on how to run your code, for example, which java file should be executed.
- Zip all the items in a folder and name as YourkfupmID\_LabSec# before submission.
- Make sure your code works. Non-working code will result in losing grades.
- Submit a PDF report containing the following:
  - Details of the data structures used. Describe where, why and how you used each data structure. Attach related code snippets from your java files to support your description.
  - Details of the algorithms used. Describe where, why, and how you used each algorithm. Attach related code snippets from your java files to support your description.
- Consider the points mentioned in the rubrics below:

| Category                                       | Details                                                                                           | Points    |
|------------------------------------------------|---------------------------------------------------------------------------------------------------|-----------|
| Questions Answers<br>(Understanding the codes) | Accurate and confident response to questions, including the ability to change the code instantly. | 7 points  |
| Team Collaboration                             | Collaborative effort.                                                                             | 3 points  |
| Performance Indicator                          | According to the following table                                                                  | 20 points |

| Performance Indicator      | Exemplary (100)                                                                                                                                                                                                                                         | Satisfactory (75)                                                                                                                                                                                                                               | Developing (50)                                                                                                                                                                                                                                           | Unsatisfactory (25)                                                                                                                                                                                                |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PI(1): Design capabilities | Design follows the requirement and considerations for space and time efficiency is evident.                                                                                                                                                             | Design follows the requirement but no considerations for space and time efficiency is evident.                                                                                                                                                  | Some parts are designed by considering the problem/requirement, but not all.                                                                                                                                                                              | No evidence of data structure and solution design based on the problem/requirement.                                                                                                                                |
| PI(2): Implementation      | <ul style="list-style-type: none"> <li>▪ Modular Code with proper encapsulation</li> <li>▪ Proper identity (variables) and behavior (methods)</li> <li>▪ Proper use of structured constructs (if, while, for, etc.)</li> <li>▪ Well indented</li> </ul> | One of the following issues: <ul style="list-style-type: none"> <li>▪ Code not modular with no proper encapsulation</li> <li>▪ Ill-structured code</li> <li>▪ Not that well indented</li> <li>▪ Some vague variable and method names</li> </ul> | More than one of the following issues: <ul style="list-style-type: none"> <li>▪ Code not modular with no proper encapsulation</li> <li>▪ Ill-structured code</li> <li>▪ Not that well indented</li> <li>▪ Some vague variable and method names</li> </ul> | <ul style="list-style-type: none"> <li>▪ Code not modular with no proper encapsulation</li> <li>▪ Ill-structured code</li> <li>▪ Not that well indented</li> <li>▪ Some vague variable and method names</li> </ul> |
| PI(3): Evaluation          | <input type="checkbox"/> Compiles without warnings or errors.<br>Runs for all cases.                                                                                                                                                                    | <input type="checkbox"/> Compiles without warnings or errors.<br>Runs for the general cases,<br>Misses at most two special cases.                                                                                                               | <input type="checkbox"/> Compiles without warnings or errors.<br>Runs for general cases.<br>At least three or more different cases do not run as per the given requirements.                                                                              | May only run for few cases or does not run/compile.                                                                                                                                                                |

=====END=====